



AI-Powered Building Damage Assessment System Using Computer Vision for Post-War Reconstruction in Gaza

by

Abdallah Iqelan
Ahmed Abu Daqa
Ali Abu Salah
Hazem Muanes
Mohammed Bashir

A dissertation submitted in partial fulfillment of the
requirements for the degree of
Bachelor of "Data science and AI" Engineering(Computer Engineering)
in the
Computer Department
of the
UCAS University, GAZA

Graduation Projects advisor:
Dr. Mohanad Abukmeil

(April 2026)

Abstract

The ongoing conflict in Gaza has caused unprecedented destruction of residential infrastructure, with approximately 62% of all housing units either partially damaged or completely destroyed. Traditional manual damage assessment methods are slow, inconsistent, and impossible to scale given the magnitude of destruction and the limited number of trained surveyors. This project proposes an AI-powered building damage assessment system that leverages computer vision to automate, standardize, and accelerate the documentation of war-induced residential damage.

The system employs a two-stage deep learning pipeline. The first stage uses YOLO12, a state-of-the-art attention-centric object detector, to identify and localize 20 indoor building element categories-including walls, ceilings, doors, windows, floors, and structural components-within images captured by standard smartphone cameras. The second stage applies a lightweight classification model based on DINOv2, EfficientNet-B3, or MobileNetV4 to classify each detected element as *Damaged* or *Undamaged*, producing a structured quantitative damage report per housing unit. Both models run entirely on-device, enabling offline operation, eliminating server-side costs, and ensuring scalability across thousands of concurrent users. Data integrity is enforced through mandatory GPS geolocation tagging at capture time, while a user-feedback loop drives iterative retraining as new annotated data is collected.

The system targets a detection accuracy of $\text{mAP@0.5} \geq 0.75$ and a classification F1 score ≥ 0.75 , evaluated on roughly 30,000 annotated indoor images. Collected data are stored in a centralized database and integrated into a data warehouse that generates statistical reports and spatial damage maps to help reconstruction authorities, humanitarian organizations, and engineering planners prioritize interventions and allocate resources. This work contributes a novel end-to-end pipeline for post-war residential damage assessment, uniquely combining multi-class indoor element detection, binary damage classification, mobile edge deployment, and a structured feedback mechanism within a single framework tailored to conflict-affected environments.

Acknowledgments

First and foremost, we express our deepest gratitude to Allah, the Almighty, for granting us the strength, patience, and perseverance to complete this work under the most difficult of circumstances.

We would like to extend our sincere appreciation to our supervisor, **Dr. Mo-hanad Abukmeil**, for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project. His expertise in artificial intelligence and deep learning has been a constant source of inspiration and direction.

We are deeply grateful to the faculty and staff of the **Computer Department** at the **University College of Applied Sciences (UCAS), Gaza**, for providing us with the academic foundation and resources necessary to undertake this research.

This project was born out of a profound sense of responsibility toward our community. We dedicate this work to the people of Gaza, whose resilience in the face of unprecedented destruction has motivated every stage of this research. We hope that this system, however modest, may one day contribute to rebuilding what has been lost.

We also wish to thank our families for their unwavering patience, encouragement, and sacrifices throughout our academic journey, particularly during the challenging conditions under which this work was carried out.

Finally, we acknowledge the open-source community and the developers of the tools and frameworks used in this project, including Ultralytics, PyTorch, Roboflow, MLflow, and DVC, whose contributions to the field of AI engineering made this work possible.

Contents

List of Tables	3
List of Figures	4
1 Introduction	5
1.1 Background	5
1.2 Problem Statement	7
1.3 Objectives	8
1.4 Project Scope	9
1.5 Project Significance	10
1.6 Report Structure	10
2 Literature Review	11
2.1 Computer Vision and Image Processing	11
2.2 Object Detection	12
2.3 The YOLO Architecture	13
2.4 Related Work	14
2.4.1 Using Computer Vision Applications for Building Crack Detection	14
2.4.2 An Adaptive Hybrid Dynamic Network for Building Damage Detection	15
2.4.3 Automated Concrete Bridge Damage Detection Using an Effi- cient Vision Transformer	15
2.4.4 Disaster Damage Detection for Buildings	16
2.4.5 Defect Detection of Prefabricated Building Components	16
2.5 Summary and Research Gap	17
3 Methodology	20
3.1 Methodology Overview	20
3.2 Data Collection and Preparation	23
3.2.1 Data Sources	23
3.2.2 Annotation	23
3.2.3 Dataset Split	24
3.2.4 Data Augmentation	24
3.3 Model 1: Element Detection with YOLO12	24
3.3.1 Architecture	24

3.3.2	Training Configuration	25
3.3.3	Validation Strategy	26
3.4	Model 2: Damage Classification	26
3.4.1	Justification for a Separate Model	26
3.4.2	Input and Output	26
3.4.3	Proposed Architecture	26
3.4.4	Training Configuration	27
3.5	Evaluation Metrics	27
3.5.1	mAP@0.5	27
3.5.2	Foundational Variables	28
3.5.3	Precision	28
3.5.4	Recall	28
3.5.5	F1 Score	28
3.5.6	Target Performance Benchmarks	29
3.6	Edge Deployment on User Devices	29
3.6.1	Concept and Motivation	29
3.6.2	Target Devices	30
3.6.3	Mobile Deployment Frameworks	30
3.6.4	Model Compression Techniques	30
3.6.5	On-Device Inference Pipeline	30
3.7	System Architecture	32
3.7.1	MLOps Toolchain	32
3.7.2	Data Layer	32
3.7.3	Data Processing Pipeline	33
3.7.4	Model Training Layer	33
3.7.5	Mobile Application Layer	33
3.7.6	Server Layer	34
3.8	Technical Stack and Project Structure	34
3.8.1	Development Environment	34
3.8.2	Recommended Project Structure	35
3.9	Work Plan and Timeline	35
3.10	Anticipated Challenges and Proposed Solutions	36

References or Bibliography

37

List of Tables

2.1	Comparison of the Proposed System with Related Work . . .	18
2.2	Performance Comparison of Recent YOLO-Based Damage Detection Models (2025 to 2026)	19
3.1	Dataset partitioning strategy	24
3.2	YOLO12 model variants	25
3.3	Confusion matrix terminology	28
3.4	Performance targets	29
3.5	On-device image quality issues and preprocessing solutions .	31
3.6	MLOps tools and their roles	32
3.7	Project phases and deliverables	35
3.8	Challenges and mitigation strategies	36

List of Figures

3.1	System architecture overview: war building damage assessment platform. This blueprint presents the complete system from an AI engineering perspective, designed for integration with mobile application, web, and data engineering teams.	22
-----	--	----

Chapter 1

Introduction

1.1 Background

The ongoing conflict in Gaza has caused extensive and unprecedented damage to the built environment, with residential housing being the most severely affected sector. According to assessments published by the World Bank and the United Nations, approximately 62% of all homes in the Gaza Strip-equivalent to nearly 291,000 housing units-were either partially damaged or completely destroyed during the analyzed period [1]. Other reports estimate that around 161,000 housing units were totally destroyed and nearly 190,000 units sustained partial damage, resulting in internal displacement of nearly 70% of Gaza's population. This massive destruction has created a complex humanitarian and engineering challenge, particularly in the context of post-war reconstruction planning.

Accurate and timely damage assessment is a critical prerequisite for effective reconstruction, prioritization of interventions, allocation of resources, and risk management. However, traditional damage documentation methods in post-conflict environments rely primarily on manual field surveys, which are slow, labor-intensive, and difficult to scale given the magnitude of destruction. Field reports indicate that assessing a single residential unit requires approximately 20-30 minutes to document damage across nearly 250 structural and non-structural elements. With fewer than 500 trained surveyors available in all governorates of Gaza, the process of collecting

comprehensive damage data for all affected buildings can take several years, significantly delaying reconstruction efforts.[1] [2]

In addition, manual surveys are prone to inconsistency between inspectors and are highly sensitive to operating conditions, accessibility constraints, and human error. In addition, reconstruction authorities and international donor organizations require accurate, verified, and tamper-resistant data to ensure transparency, prevent fraud, and support fair distribution of aid. The absence of automated verification mechanisms increases the risk of inconsistent or manipulated damage records, thus undermining trust in the reconstruction process and delaying evidence-based decision-making.

Recent advances in **Artificial Intelligence(AI)**, particularly in computer vision and deep learning, provide promising opportunities to automate and standardize damage assessment workflows. Deep learning models have demonstrated strong performance in tasks such as object detection, damage classification, and semantic segmentation of building components, enabling faster and more consistent analysis compared to traditional manual approaches [3, 4]. Automated image-based analysis can generate structured, quantitative damage information at scale while supporting, rather than replacing, expert judgment in engineering assessments.

To address these challenges, this project proposes the development of an intelligent, AI-driven damage assessment system tailored to residential buildings affected by war-related destruction in Gaza. The proposed system enables residents to register their housing units and capture images of each room using standard smartphone cameras, while automatically recording geolocation metadata via GPS to ensure data integrity and prevent manipulation. A trained deep learning model analyzes the captured images to detect, classify, and quantify visible damage across building components such as walls, ceilings, openings, and structural elements.

All collected data are stored in a centralized database and integrated into a data warehouse designed to generate statistical insights and spatial damage maps. These outputs aim to support reconstruction authorities, engineers, and humanitarian organizations in prioritizing interventions, allocating resources efficiently, and planning reconstruction strategies based on reliable, evidence-based information. The proposed system does not replace comprehensive structural safety evaluations or habitability

assessments; instead, it functions as a scalable, standardized, and transparent preliminary tool that bridges the gap between large-scale destruction and expert-led reconstruction planning.

1.2 Problem Statement

Post-war residential reconstruction in Gaza faces a critical challenge related to the efficient, accurate, and trustworthy assessment of housing damage caused by war-related destruction. Although some computerized tools exist to support manual documentation and visual inspection, current damage assessment practices still rely predominantly on human-based field surveys and non-standardized interpretation of visual data. This reliance creates four primary challenges:

1. **Scalability:** Manual inspection and documentation require substantial human effort and time, making it impractical to rapidly assess hundreds of thousands of damaged residential units under conditions of limited personnel and restricted access. Each housing unit may require the documentation of hundreds of damaged structural and non-structural elements, which significantly slows down the assessment process and delays reconstruction planning.
2. **Reliability:** Human-based assessments are highly sensitive to individual judgment, varying documentation practices, and access constraints, leading to non-uniform and sometimes unreliable damage records. This variability reduces data comparability and weakens the reliability of damage datasets used by engineers, planners, and decision-makers.
3. **Automation:** There is a lack of structured automation in current damage assessment workflows. Limited integration of computer vision techniques, such as object detection, damage localization, and image-based classification, exists within systematic data pipelines capable of producing structured, machine-readable outputs for further analysis [5]. As a result, available damage data are often fragmented, unstandardized, and difficult to integrate into large-scale reconstruction planning systems.

4. **Integrity:** The absence of automated verification mechanisms increases the risk of data manipulation, exaggerated damage claims, and fraudulent reporting. This undermines transparency and negatively affects decision-making by reconstruction authorities and international donor organizations.
5. **Maintainability and Continuous Improvement:** Unlike traditional data collection methods, which rely heavily on human effort and become increasingly prone to errors as data volume grows, the proposed system is designed for long-term maintainability and continuous performance improvement. The application enables systematic monitoring of detection errors in the field, which are reviewed by the AI engineering team and used to fine-tune the model through iterative retraining cycles. This feedback-driven improvement loop ensures that detection accuracy increases progressively as more images are collected and annotated, a characteristic that is particularly critical in the context of Gaza, where precise and reliable damage detection directly supports evidence-based post-war reconstruction planning.

Therefore, the core problem addressed in this project is the lack of an intelligent, automated, and scalable image-based system capable of detecting, localizing, and classifying visible damage patterns in residential buildings in Gaza using computer vision techniques, while ensuring data integrity and consistency. The proposed system aims to generate structured and reliable damage information that supports engineers, planners, and authorities in prioritizing further inspections and reconstruction actions. It is important to note that this system does not perform full structural safety assessments or certify habitability; rather, it serves as a preliminary decision-support tool that bridges the gap between large-scale destruction and expert-led reconstruction workflows.

1.3 Objectives

The main objectives of this project, defined according to the SMART criteria, are as follows:

- Develop an (AI), based system capable of automatically detecting and assessing war-related damage in residential buildings in Gaza using images and computer vision techniques.
- Reduce the time required for damage data collection per residential unit by automating the detection and classification process through image-based AI assessment.
- Achieve a detection accuracy of at least 85% using Deep learning models, and reliability of housing damage assessment data through geolocation-based verification mechanisms such as GPS.
- Support reconstruction authorities and humanitarian organizations by generating accurate statistical reports derived from damage data automatically extracted by the computer vision model, enabling effective prioritization of housing reconstruction efforts and efficient allocation of limited resources.

1.4 Project Scope

The scope of this project includes:

- Image collection using different imaging capture tools and mobile devices.
- Development and training of AI technologies including deep learning models (e.g., object detection or classification architectures) for visible damage identification.
- Data storage and basic analytical reporting of detected and classified damage elements.

This project does not include:

- Full structural engineering assessment or safety evaluation,
- Certification of building habitability,
- Physical reconstruction efforts, procurement of materials, or legal claims processing.

1.5 Project Significance

The significance of this proposed project lies in its contribution to the application of AI and computer vision in post-conflict infrastructure analysis in Gaza:

- It demonstrates how modern AI- powered object detection and image classification techniques can be applied to real-world damage detection and documentation problems.
- It provides a structured prototype for scaling image-based damage detection across large areas.
- It contributes to the growing body of work on automated analysis of built environments affected by conflict or disaster, with potential applicability in other contexts such as earthquakes and hurricanes (Kim et al 2022).

1.6 Report Structure

The report is structured into five chapters. Following this introduction.

Chapter 2 reviews related work and existing damage assessment approaches.

Chapter 3 presents the proposed system architecture and methodology.

Chapter 2

Literature Review

The use of artificial intelligence in image analysis has evolved significantly over the past two decades, progressing from basic image processing techniques to sophisticated deep learning architectures capable of understanding complex visual scenes. This chapter traces that evolution, narrowing progressively from the broad field of computer vision through object detection to the YOLO family of models, and finally to the specific body of recent work most relevant to this project. The chapter concludes with a comparative analysis that positions the proposed system in relation to existing approaches and identifies the research gap it addresses.

2.1 Computer Vision and Image Processing

Computer vision is the field of artificial intelligence concerned with enabling machines to interpret and understand visual information from the world. Its foundations lie in digital image processing, which emerged in the 1960s alongside early space exploration programs that required automated enhancement and analysis of satellite imagery [3]. Early techniques focused on pixel-level operations such as filtering, edge detection, and thresholding, mathematical transformations that extract structural features from raw image data.

The field advanced substantially through the 1980s and 1990s with the development of feature descriptors such as SIFT (Scale-Invariant Feature Transform) and

HOG (Histogram of Oriented Gradients), which enabled machines to recognize objects based on local patterns rather than individual pixels. These handcrafted features, however, required significant domain expertise to design and struggled to generalize across diverse visual conditions.

The introduction of deep learning, and convolutional neural networks (CNNs) in particular, transformed the field. CNNs learn hierarchical feature representations directly from training data, eliminating the need for manual feature engineering. The landmark AlexNet architecture, which won the ImageNet Large Scale Visual Recognition Challenge in 2012, demonstrated that deep CNNs could dramatically outperform traditional methods on image classification tasks. This breakthrough catalyzed a wave of research that rapidly extended deep learning to more complex visual tasks, including object detection, semantic segmentation, and damage assessment [4].

2.2 Object Detection

Object detection is a computer vision task that requires both identifying what objects are present in an image and localizing them by predicting their bounding boxes. This dual requirement, classification and localization simultaneously, makes object detection substantially more challenging than image classification alone.

Early deep learning approaches to object detection followed a two-stage paradigm. Region-based CNN (R-CNN) and its successors Fast R-CNN and Faster R-CNN first generated candidate regions likely to contain objects, then classified each region independently. While accurate, these methods were computationally expensive and unsuitable for real-time applications.

Single-stage detectors addressed this limitation by reframing detection as a single regression problem: rather than proposing candidate regions, the model predicts bounding boxes and class probabilities directly from the full image in one forward pass. This approach offered substantially faster inference at a modest cost in accuracy. The SSD (Single Shot MultiBox Detector) architecture exemplified this approach and demonstrated that real-time detection was achievable with deep learning [5].

The performance gap between single-stage and two-stage detectors has narrowed significantly in recent years, with modern single-stage architectures matching or exceeding the accuracy of earlier two-stage models while maintaining their speed advantage. This evolution made single-stage detection the dominant paradigm for applications requiring real-time or on-device inference, including the damage assessment system proposed in this project.

2.3 The YOLO Architecture

YOLO (You Only Look Once), introduced by Redmon et al. [6], became the defining architecture in real-time single-stage object detection. The core innovation of YOLO was to treat detection as a unified regression problem: the input image is divided into a grid, and each grid cell simultaneously predicts bounding boxes, objectness scores, and class probabilities. This design enabled detection in a single network pass, achieving speeds far beyond those of two-stage methods while maintaining competitive accuracy.

Since its introduction, YOLO has undergone continuous architectural evolution through successive versions, each improving upon its predecessor in accuracy, speed, and efficiency. YOLOv3 introduced multi-scale detection using feature pyramid networks. YOLOv4 incorporated extensive training augmentations and architectural improvements. YOLOv5 refined the implementation for practical deployment. YOLOv7 and YOLOv8 introduced architectural innovations that significantly improved detection across object sizes.

The release of YOLOv11 in late 2024 marked a significant architectural leap, introducing an attention-centric design that integrates attention mechanisms directly into the detection backbone. This allowed the model to better capture long-range spatial dependencies without sacrificing inference speed. YOLOv11's improved feature pyramid network and enhanced multi-scale detection capabilities made it particularly suitable for detecting objects of varying sizes, including the large diversity of indoor building elements targeted in this project.

YOLO12, the version adopted in this project, extends this attention-centric paradigm further. Its architecture achieves a more balanced trade-off between accuracy and computational efficiency, and is specifically designed to support deployment on resource-constrained devices including mobile phones, a critical requirement for the proposed field-deployable damage assessment system.

2.4 Related Work

The rapid evolution of YOLO architectures since late 2024 has enabled a new wave of research applying these models to structural damage detection. The five studies reviewed in this section were selected based on three criteria: recency (published between 2025 and 2026), architectural relevance (YOLO-based or attention-based detection), and contextual proximity (damage detection in built environments). Each study is examined for both its contributions and its limitations in relation to the specific challenge of large-scale indoor residential damage assessment.

2.4.1 Using Computer Vision Applications for Building Crack Detection

An improved YOLOv11 model specifically designed for building crack detection was presented, addressing persistent challenges in computer vision applications for structural health monitoring [7]. Gao et al. integrated three key architectural modifications: a C3K2-SG module for enhanced feature extraction, an FPSConv module to reduce computational overhead while maintaining detection accuracy, and an Inner MPDIoU loss function to improve bounding box regression for small, irregular crack patterns. Tested on diverse crack datasets, the model achieved mAP@0.5 of 88.6%, representing a 4.6% improvement over the baseline, with Precision and Recall improving by 3.5% and 7.6% respectively. However, this study focuses exclusively on a single damage type, surface cracks, and does not address the broader challenge of detecting and classifying damage across diverse indoor building elements. No binary damage classification pipeline is proposed, and the system is not evaluated for mo-

ble deployment, which is a core requirement of the proposed system given the field conditions in Gaza.

2.4.2 An Adaptive Hybrid Dynamic Network for Building Damage Detection

AHD-YOLO (Adaptive Hybrid Dynamic YOLO), a model designed explicitly for identifying structural damage in conflict-affected urban environments, was proposed by Li et al. [8]. The architecture combines YOLOv11 with an adaptive fusion strategy that dynamically adjusts to the highly variable nature of conflict-related damage, ranging from precision-strike damage to widespread rubble fields. The model incorporates a hybrid attention mechanism that balances channel and spatial attention based on the density and distribution of detected damage patterns. Tested on datasets from recent conflicts, including imagery from Ukraine and Gaza, AHD-YOLO achieved a mean Average Precision of 70.5%, outperforming YOLOv11s by 0.6% while reducing model parameters by 11%. While this study is the most contextually relevant to the Gaza setting, it addresses damage at the building level from aerial or street-level perspectives rather than at the indoor element level. It does not incorporate a two-stage detection and classification pipeline, does not target deployment on user-owned mobile devices, and does not address the specific challenge of identifying 20 indoor structural and non-structural elements.

2.4.3 Automated Concrete Bridge Damage Detection Using an Efficient Vision Transformer

The integration of Vision Transformers with anchor-free YOLO architectures for automated concrete bridge damage detection was explored by Yang et al. [9]. Their model, built on a YOLOv5l backbone, replaces standard C3 modules with C3MaxViT blocks, which incorporate multi-axis self-attention mechanisms to capture long-range spatial dependencies in high-resolution bridge imagery. The model achieved an improvement of 8.1% in mAP@0.5 compared to the baseline YOLOv5l model, with no-

table improvements in detecting low-contrast cracks on weathered concrete surfaces. The work reflects a broader trend toward hybrid CNN/Transformer architectures for infrastructure inspection tasks where both local texture detail and global structural context matter. This study demonstrates the value of attention mechanisms for damage detection but is limited to a single infrastructure type (bridges) and a single damage category. The model’s high computational requirements make it unsuitable for mobile deployment, and no damage classification component is provided.

2.4.4 Disaster Damage Detection for Buildings

DDNet (Disaster Damage Detection Network), a dual-temporal joint attention network designed to assess building damage by comparing pre- and post-disaster imagery, was introduced by Ge et al. [10]. DDNet leverages temporal change information using a dual-branch architecture to extract features from both time points and a joint attention module to highlight regions where structural changes have occurred. Tested on datasets from earthquakes and typhoons, DDNet achieved an F1 score of 79.56% with localization accuracy of 86.38%. This temporal comparison framework effectively reduces false positives by distinguishing new damage from pre-existing conditions. Despite its novelty, DDNet requires paired pre- and post-disaster images, which are unavailable in many real-world scenarios including Gaza, where systematic pre-conflict indoor imagery does not exist at scale. The system operates at the building level rather than the element level and is not designed for on-device inference.

2.4.5 Defect Detection of Prefabricated Building Components

A YOLOv5s-GhostNet hybrid for defect detection in prefabricated building components was proposed by Liu et al. [11], achieving 99% crack detection accuracy while maintaining compatibility with mobile and embedded devices. Similarly, a lightweight YOLOv11n-Seg framework introduced real-time segmentation capabilities for surface crack detection, enabling pixel-level damage mapping on smartphones and drones without requiring cloud connectivity. These developments underscore a key theme

in the literature: for damage detection systems to be practically useful, they must balance accuracy with computational efficiency. While these works demonstrate the feasibility of lightweight deployment, they target narrow defect categories in controlled manufacturing or construction contexts, not the diverse and visually complex damage patterns encountered in post-war residential buildings. Neither system incorporates a multi-class element detection stage or a binary damage classification pipeline driven by user feedback.

2.5 Summary and Research Gap

The five studies reviewed in this chapter collectively demonstrate meaningful progress in AI-driven damage detection. Gao et al. [7] advanced crack detection precision through targeted architectural modifications to YOLOv11. Li et al. [8] developed a model specifically adapted to the visual complexity of conflict-zone damage. Yang et al. [9] showed how hybrid CNN-Transformer architectures improve detection of distributed damage patterns in infrastructure. Ge et al. [10] demonstrated the value of temporal change analysis for reducing false positives in post-disaster surveys. Liu et al. [11] established the feasibility of accurate, lightweight detection on mobile and embedded devices.

Despite this progress, a consistent gap emerges across all five studies: none addresses the specific challenge of systematically detecting and classifying damage across a large vocabulary of indoor building elements, including walls, ceilings, doors, windows, furniture, appliances, and structural components, in the context of residential buildings damaged by armed conflict. Table 2.1 summarizes the key dimensions along which the proposed system differs from the reviewed work.

Table 2.1: Comparison of the Proposed System with Related Work

Criterion	Gao (2025)	Li (2026)	Yang (2025)	Ge (2025)	Liu (2025)	Proposed System
Indoor element detection (20 classes)	No	No	No	No	No	Yes
Binary damage classification per element	No	No	No	No	No	Yes
Two-stage pipeline	No	No	No	No	No	Yes
Mobile deployment on user device	No	No	No	No	Partial	Yes
Works without pre-disaster imagery	Yes	Yes	Yes	No	Yes	Yes
Conflict-zone / war damage context	No	Partial	No	No	No	Yes (Gaza)
User feedback loop for retraining	No	No	No	No	No	Yes
Image type	Close-up ground	Ground + aerial	High-res ground	Satellite	Laboratory	Indoor smart-phone
Viewpoint	Exterior	Exterior	Exterior	Aerial	Exterior	Interior

The proposed system addresses this gap by combining a YOLO12-based element detection model with a binary damage classification model in a two-stage pipeline, deployed entirely on the user’s mobile device. Unlike all reviewed works, it targets 20 indoor building element classes, incorporates a structured user feedback mechanism to support continuous model improvement, and is designed specifically for the post-war residential damage context of Gaza.

It is important to note that additional studies exist in the broader field of AI-based structural damage detection. The five works reviewed here were selected for their direct relevance to the objectives of the proposed project and their focus on recent YOLO-based architectural developments from 2025 to 2026 Table 2.2. Other works, although valuable, address different problem contexts and fall outside the specific scope of this review.

Table 2.2: Performance Comparison of Recent YOLO-Based Damage Detection Models (2025 to 2026)

#	Study	YOLO Variant	Damage Type	Key Contribution	Performance
1	Gao et al. [7]	YOLOv11	Building cracks	C3K2-SG module + FP-SCConv + Inner-MPDIoU loss	mAP@0.5 = 88.6% (+4.6% vs. baseline), Precision +3.5%, Recall +7.6%
2	Li et al. [8]	YOLOv11 (AHD-YOLO)	Conflict-zone damage	Adaptive hybrid dynamic network with attention fusion	mAP: 70.5%; 11% parameter reduction
3	Yang et al. [9]	YOLOv5l + ViT	Bridge concrete damage	C3MaxViT blocks with multi-axis self-attention	+8.1% mAP@0.5 over baseline
4	Ge et al. [10]	Custom (DDNet)	Post-disaster buildings	Dual-temporal joint attention for change detection	F1 = 79.56%; Localization = 86.38%; Classification = 76.64%
5	Liu et al. [11]	YOLOv5s-GhostNet	Prefabricated components	GhostNet integration for mobile deployment	99% crack detection accuracy

In the next chapter, Chapter 3, the proposed methodology is presented in detail, covering the system architecture, data acquisition and preprocessing workflow, model design and training configuration, evaluation metrics, and deployment strategy.

Chapter 3

Methodology

3.1 Methodology Overview

This research adopts an applied research methodology based on AI engineering principles, with object detection as its core technical foundation, centered on the development of a two-stage AI-driven system for the automated detection and classification of war-induced building damage. The methodology covers the full project lifecycle, encompassing data collection, model training, performance evaluation, and deployment on the users' own devices through a mobile application.

The system comprises two complementary models operating in a sequential pipeline:

- **Model 1: Element Detection:** A YOLO12-based object detection model trained to identify 20 indoor building elements within damage scene images.
- **Model 2: Damage Classification:** A visual classification model that receives an image crop containing a detected element and classifies its condition as *Damaged* or *Undamaged*.
- **Edge Deployment:** Both models run entirely on the user's device, enabling field operation without internet connectivity, eliminating server-side processing costs, and reducing server load.

The system architecture presented in Figure 3.1 serves as a comprehensive engineering blueprint that enables all project stakeholders to visualize the complete system and understand how each component fits within the whole. This design allows

every specialist to focus on their own domain while maintaining a shared understanding of the overall structure: AI engineers handle model development, training, and monitoring; mobile application developers implement the on-device inference engine and user interface; web developers build the backend server and administrative portal; and data engineers design the Data Warehouse schema and ETL workflows in greater depth.

From an AI engineering perspective, this architecture defines the core requirements for data collection, model training, deployment, and continuous monitoring. It is important to note that this blueprint reflects the system requirements as seen through the lens of AI engineering and may not capture all technical requirements specific to other disciplines. Each specialization is expected to extend and refine the relevant components according to their own domain standards and constraints. This architecture is designed for integration, providing a clear contract between all disciplines so that each team can work independently within their domain while ensuring that the outputs of one component align precisely with the inputs of another.

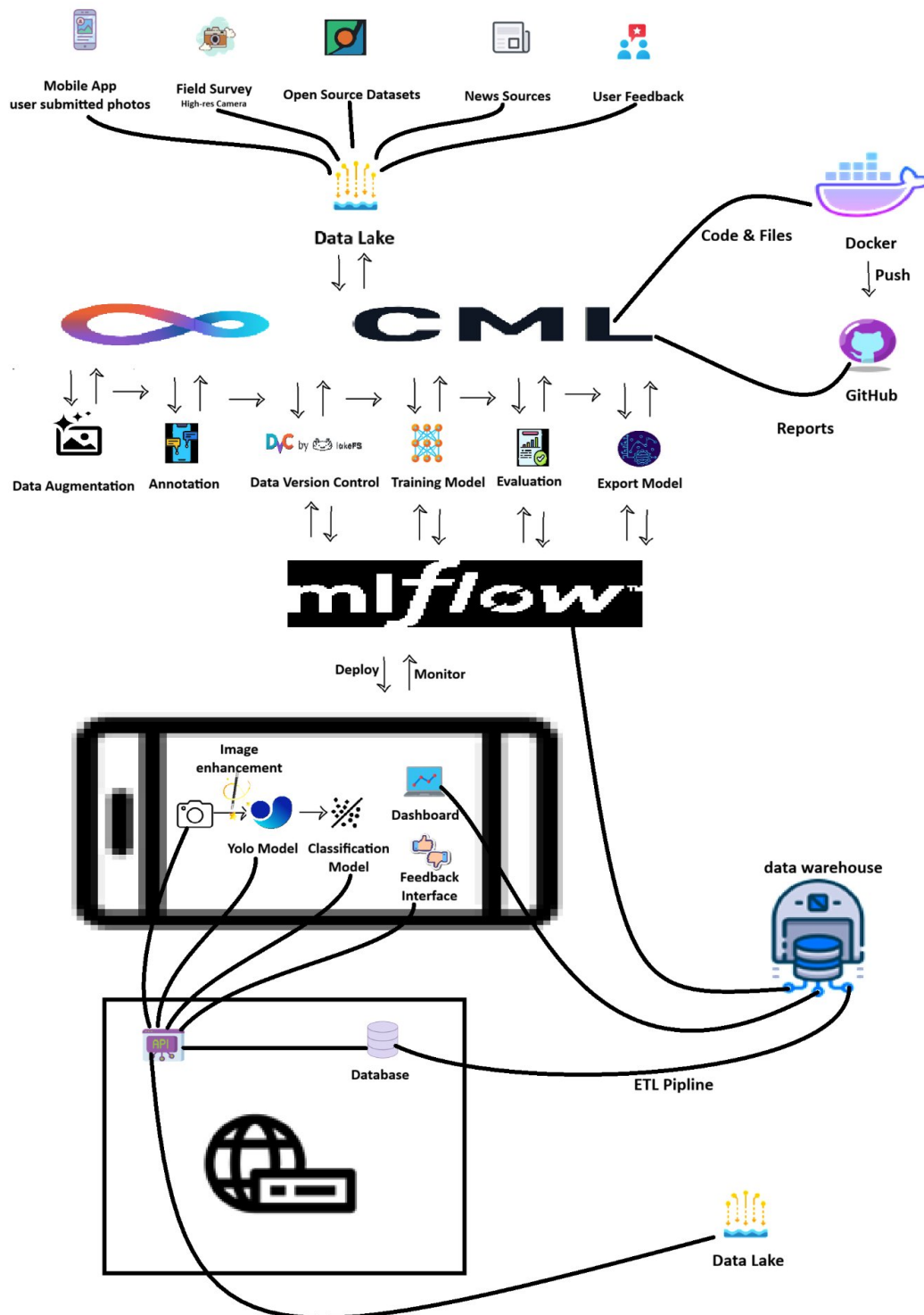


Figure 3.1: **System architecture overview: war building damage assessment platform.** This blueprint presents the complete system from an AI engineering perspective, designed for integration with mobile application, web, and data engineering teams.

3.2 Data Collection and Preparation

3.2.1 Data Sources

Image data may be sourced from one or more of the following origins, depending on availability and accessibility during the data collection phase. Not all sources are required; the system is designed to function effectively with any combination of the following:

- **Mobile Application:** Photos submitted by users through the application.
- **Field Survey:** High-resolution images captured directly in damaged areas.
- **Open-Source Datasets:** Publicly available collections relevant to building damage.
- **News Sources:** Images from media outlets collected with appropriate licensing.

All images are uploaded to a centralized Data Lake and organized by source, timestamp, and geographic metadata where available. Feedback images submitted by users are annotated by the AI engineering team and incorporated into future training cycles.

3.2.2 Annotation

The annotation phase adheres to the following standards:

- **Annotation Tool:** Roboflow or CVAT for drawing bounding boxes around each element instance.
- **Export Format:** YOLO Format (.txt) with normalized coordinates.
- **Detection Classes:** 20 indoor building element categories, defined as follows: Brick/Wall, Column, Staircase, Floor Tiles, Ceiling, Beam, Kitchen Counter-top, Sink, Wall Cabinet, Chair, Floor Cabinet, Bathtub, Water Faucet, Window, Fire Extinguisher, Roof, Door, Air Conditioner, Window Sill, and Light Fixture.
- **Classification Labels:** Binary labels per detected crop: Damaged (1) / Undamaged (0).

- **Annotation Team:** Multi-annotator review per image with Inter-Annotator Agreement (IAA) measured using Cohen’s Kappa.

3.2.3 Dataset Split

Table 3.1: Dataset partitioning strategy

Split	Ratio	Purpose	Approx. Size
Training Set	70%	Model optimization	~21,000 images
Validation Set	15%	Hyperparameter tuning	~4,500 images
Test Set	15%	Final evaluation	~4,500 images

3.2.4 Data Augmentation

To improve generalization and mitigate data scarcity, the following augmentation techniques may be fully or partially applied during training, depending on the dataset characteristics and model performance:

- Horizontal and vertical flip.
- Random rotation ($\pm 15^\circ$).
- Brightness and contrast jitter.
- Gaussian noise and blur.
- Mosaic Augmentation (built into YOLO).
- Random crop and scale.
- MixUp Augmentation.

3.3 Model 1: Element Detection with YOLO12

3.3.1 Architecture

YOLO12 (You Only Look Once: Version 12) represents the state of the art in real-time object detection. Its Attention-Centric architecture integrates attention

mechanisms directly into the backbone, enabling higher localization accuracy without sacrificing inference speed. Multiple model variants are available to balance accuracy against computational constraints of target mobile devices. The mobile-compatible variants (YOLO12n, YOLO12s, and YOLO12m) will be trained and evaluated based on the available GPU resources of the development team, with the most suitable variant selected for final deployment:

Table 3.2: **YOLO12 model variants**

Variant	Parameters	FLOPs	mAP@0.5	Target Device
YOLO12n	~2.6M	~6.5G	Moderate	Low-end phones
YOLO12s	~9.3M	~21.5G	Good	Mid-range phones
YOLO12m	~20M	~67G	High	High-end phones
YOLO12l	~26M	~89G	Very High	Training servers
YOLO12x	~59M	~199G	Highest	High-end servers

3.3.2 Training Configuration

- **Framework:** Ultralytics YOLO12 with PyTorch backend.
- **Input Resolution:** 640×640 px (extendable to 1280 for improved small-object detection).
- **Batch Size:** 16 or 32, depending on available GPU memory.
- **Learning Rate:** 0.01 with Cosine Annealing Scheduler.
- **Optimizer:** SGD with momentum=0.937, or AdamW.
- **Epochs:** 100-300 with Early Stopping (patience=50).
- **Weight Initialization:** Transfer Learning from pretrained weights.
- **Loss Function:** Box Loss + Classification Loss + DFL Loss.
- **Inference Parameters:** Confidence Threshold and IoU Threshold tuned to balance accuracy and speed on mobile devices.

3.3.3 Validation Strategy

The detection model is evaluated on the validation set during training (per-epoch mAP monitoring) and on the held-out test set for final reporting across all 20 classes.

3.4 Model 2: Damage Classification

3.4.1 Justification for a Separate Model

Two independent models are selected for the following reasons:

- **Data scarcity:** Separating the models allows each to be trained independently on its optimal dataset without conflict.
- **Independent updates:** When new feedback data arrives, only the classification model is retrained without affecting the detection model.
- **Resource efficiency:** The two models run sequentially on the device, maintaining acceptable performance on mobile hardware.

3.4.2 Input and Output

The classification model receives a single input per detected element:

- **Image Crop:** The region extracted from the original image using the bounding box predicted by YOLO12, resized to 256×256 px.

The image crop alone is sufficient for damage assessment, as damage is inherently visual, cracks, burns, deformation, breakage, patterns the model learns directly from the crop without additional input.

Output: Binary classification: Damaged (1) / Undamaged (0) with a calibrated confidence score.

3.4.3 Proposed Architecture

The model adopts a lightweight visual classification architecture suitable for mobile deployment:

- **Backbone:** Dinov2, EfficientNet-B3 or MobileNetV4, pretrained on ImageNet with fine-tuning on damage-specific data.
- **Head:** Fully Connected layers with Dropout=0.3.
- **Output Layer:** Sigmoid activation for binary classification.

3.4.4 Training Configuration

- **Loss Function:** Binary Cross-Entropy (BCE) with class weighting to handle class imbalance.
- **Optimizer:** AdamW with learning rate 10^{-4} .
- **Batch Size:** 32-64.
- **Epochs:** 30-80 with Early Stopping.
- **Imbalance Handling:** Class weights or Focal Loss when the damaged/undamaged ratio is highly skewed.

3.5 Evaluation Metrics

3.5.1 mAP@0.5

The standard benchmark for object detection, computing the mean Average Precision across all classes at IoU threshold 0.5:

$$\text{mAP@0.5} = \frac{1}{N} \sum_{i=1}^N AP_i \quad (N = 20)$$

Additionally, mAP@0.5:0.95 is reported as a stricter supplementary metric.

3.5.2 Foundational Variables

Table 3.3: **Confusion matrix terminology**

Symbol	Term	Definition
TP	True Positives	Elements correctly detected and/or classified.
FP	False Positives	Elements incorrectly predicted.
FN	False Negatives	Existing elements missed by the model.
TN	True Negatives	Non-existent elements correctly ignored.

3.5.3 Precision

Measures the proportion of correct predictions among all positive predictions:

$$\text{Precision} = \frac{TP}{TP + FP}$$

A high Precision indicates a low false alarm rate.

3.5.4 Recall

Measures the proportion of actual positives successfully identified:

$$\text{Recall} = \frac{TP}{TP + FN}$$

A high Recall ensures that genuine damage instances are not overlooked.

3.5.5 F1 Score

The harmonic mean of Precision and Recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

F1 is the primary metric when class imbalance is present.

3.5.6 Target Performance Benchmarks

Table 3.4: Performance targets

Metric	Model 1 (YOLO12)	Model 2 (Classifier)	Minimum
mAP@0.5	≥ 0.75	—	0.75
Precision	≥ 0.80	≥ 0.85	0.80
Recall	≥ 0.71	≥ 0.76	0.71
F1 Score	≥ 0.75	≥ 0.80	0.75

3.6 Edge Deployment on User Devices

3.6.1 Concept and Motivation

The system runs entirely on the user’s smartphone. The user installs the application, photographs their building, and the on-device models perform the full assessment locally. This design achieves:

- **Offline operation:** Inference requires no internet connection. Results are stored locally and synchronized to the server when connectivity is available.
- **Zero processing cost:** Regardless of the number of images processed, thousands or millions, all computation runs on users’ own devices at no cost to the development team.
- **Reduced server load:** The server receives structured results rather than raw images for processing, improving response times and reducing infrastructure demands.

3.6.2 Target Devices

- Android smartphones (mid-range and high-end).
- iOS smartphones (recent iPhone models).

3.6.3 Mobile Deployment Frameworks

- **TensorFlow Lite (TFLite):** Suitable for Android and iOS, supports INT8 and FP16 quantization.
- **ONNX Runtime Mobile:** Cross-platform compatibility with high performance.
- **Core ML:** Optimized for Apple devices, leverages the Neural Engine.

3.6.4 Model Compression Techniques

- **Post-Training Quantization (PTQ):** Convert weights from FP32 to INT8, 4× size reduction with minimal accuracy loss.
- **Quantization-Aware Training (QAT):** Simulate quantization during training to preserve accuracy.
- **Model Pruning:** Remove low-magnitude weights to reduce complexity.
- **Knowledge Distillation:** Train a compact Student model to mimic a larger Teacher model.

3.6.5 On-Device Inference Pipeline

1. **Capture image and record location:** The user captures an image directly using the device camera. Uploading images from the gallery is not permitted, as this would undermine data integrity and prevent accurate geolocation tagging. GPS is activated simultaneously to record the geographic coordinates of the captured image, ensuring that each assessment is traceable to its physical location and resistant to fraudulent or manipulated submissions.
2. **Image Enhancement:** Before running inference, lightweight preprocessing techniques are applied to improve the input image quality and maximise de-

tection accuracy. Since the models were trained on raw, unenhanced images to ensure robustness, enhancement at inference time provides additional clarity without introducing a domain mismatch. Table 3.5 summarises the quality issues addressed and their corresponding solutions.

Table 3.5: **On-device image quality issues and preprocessing solutions**

Issue	Cause in This Project	Preprocessing Solution
Motion blur	Handheld capture by non-expert users without stabilization	Sharpening filters; Wiener de-blurring for severe cases
Low brightness	Insufficient lighting in damaged rooms	CLAHE (Contrast Limited Adaptive Histogram Equalization)
Overexposure	Direct sunlight through broken windows or openings	Gamma correction to compress highlights
Noise	Low-quality sensors in mid-range smartphones	Gaussian and median filtering for noise suppression
Low resolution	Variation in smartphone camera specifications	Bicubic interpolation for resolution normalization
Lens distortion	Wide-angle lenses used in confined spaces	OpenCV-based radial distortion correction
Color cast	Unusual lighting conditions in damaged environments	White balance correction and color channel normalization
Partial occlusion	Debris and rubble partially covering elements	Addressed through Random Occlusion augmentation during training

3. **Run YOLO12** (TFLite/ONNX Mobile), detect elements and generate bounding boxes.
4. **Crop** image regions based on YOLO12 predictions.
5. **Run the classification model** on each crop: Damaged / Undamaged.
6. **Store results** locally; display on dashboard; sync to server when connected.

3.7 System Architecture

As introduced in Section 3.1, the system architecture serves as a multi-disciplinary engineering blueprint ready for integration with all relevant specializations. The system consists of three main parts operating in an integrated manner: the mobile application, the web application (server), and the MLOps pipeline. Figure 3.1 presents the detailed component view and full data flow from sources to the dashboard.

3.7.1 MLOps Toolchain

Table 3.6: MLOps tools and their roles

Tool	Role
Docker	Containerizes all services and training environments for reproducibility.
GitHub Actions	Automates CI/CD pipelines on every code commit or pull request.
CML	Generates evaluation reports automatically as pull request comments.
DVC	Tracks dataset versions and links each experiment to its exact data snapshot.
MLflow	Logs all experiments and manages the model registry (Staging → Production).

3.7.2 Data Layer

A centralized Data Lake stores all raw imagery from all sources: mobile application photos, field survey images, open-source collections, and news imagery. Images are organized by source, timestamp, and geographic metadata. Feedback images from users are also routed to the Data Lake, linked to their database records by full file path.

3.7.3 Data Processing Pipeline

Raw images are stored in the Data Lake as collected, without prior quality enhancement. This is intentional: training the models on realistic images that contain natural noise, blur, and variable lighting conditions ensures that both models generalise effectively to the diverse field conditions they will encounter during deployment. Artificially cleaning the training data would produce models that perform well on pristine images but fail on the real-world imagery submitted by users.

Raw images pass through the following sequential stages before release as a versioned training dataset:

1. **Annotation:** Bounding-box labels for 20 element classes and binary damage labels per crop using CVAT or Roboflow.
2. **Dataset Versioning:** Published via DVC and made available for training.

3.7.4 Model Training Layer

Versioned datasets feed the training pipeline:

- Model 1 (YOLO12) and Model 2 (EfficientNet classifier) are trained in isolated Docker containers.
- All experiments, hyperparameters, metrics, curves, and weights, are logged to MLflow.
- Dataset versions are tracked via DVC for full reproducibility.
- CML posts automated evaluation reports to GitHub pull requests.
- Upon passing quality thresholds, the best checkpoint is promoted to Production in the MLflow registry and exported for mobile deployment.
- Performance is monitored in the field; retraining is triggered automatically on degradation.

3.7.5 Mobile Application Layer

The application runs on the user's device and contains:

- On-device inference engine (Model 1 + Model 2, compressed for mobile).

- Image capture interface.
- Results dashboard.
- Feedback interface for flagging model errors.

Data flow: the image capture interface sends images to the server for storage and to Model 1 for detection. Model 1 results flow to Model 2 for classification and to the server. Model 2 results flow to the server. All transfers are asynchronous, data is saved locally and synced when internet is available.

3.7.6 Server Layer

The backend server exposes a REST API and manages all data persistence:

- **Result Ingestion:** Structured JSON payloads from the app are stored in a relational database.
- **Image Storage:** Captured images are routed to the Data Lake. Each record stores the full Data Lake path for traceability.
- **Feedback Processing:** Corrections are stored with the image filename and full path, then flagged for the data engineering team.
- **Data dispatch:** Approved and updated data is sent back to the mobile application via API.
- **Web Portal:** Browser interface for administrators and stakeholders.

3.8 Technical Stack and Project Structure

3.8.1 Development Environment

- Python 3.10+
- PyTorch 2.x with CUDA 12.x
- Ultralytics YOLO12 (`ultralytics >= 8.x`)
- TensorFlow Lite, ONNX Runtime Mobile, Core ML
- MLflow, Git, DVC
- GitHub Actions, Docker, CML

3.8.2 Recommended Project Structure

```
war_damage_detection/  
├── data/           (raw, processed, annotations)  
├── models/         (detection, classification, exported)  
├── training/       (train_det.py, train_cls.py)  
├── evaluation/     (metrics.py, confusion_matrix.py)  
├── edge/           (quantize.py, export_tflite.py, export_onnx.py)  
└── pipeline/      (end2end_inference.py)
```

3.9 Work Plan and Timeline

The project is structured into six sequential phases, each with clearly defined tasks, deliverables, and estimated durations. Table 3.7 presents the planned timeline, which is subject to adjustment based on data availability, computational resources, and experimental outcomes encountered during execution.

Table 3.7: Project phases and deliverables

#	Phase	Key Tasks	Deliverables	Est. Duration
1	Data Collection & Annotation	Image sourcing, bounding box annotation, quality review	Annotated dataset (~30,000 images)	4 weeks
2	YOLO12 Training	Hyperparameter tuning, 20-class training, mAP evaluation	Optimized detection model	3 weeks
3	Classifier Development	EfficientNet/MobileNet training, binary evaluation	Damage classification model	2 weeks
4	Pipeline Integration	End-to-end pipeline assembly, full system testing	Working end-to-end system	2 weeks
5	Mobile Deployment Preparation	TFLite/ONNX export, quantization, on-device testing	Mobile-ready model packages	2 weeks
6	Final Evaluation & Documentation	Comprehensive testing, paper writing	Research report	3 weeks

3.10 Anticipated Challenges and Proposed Solutions

Several technical and operational challenges are anticipated during the development of this system. Table 3.8 identifies each challenge along with the corresponding mitigation strategy adopted in this project.

Table 3.8: **Challenges and mitigation strategies**

Challenge	Proposed Solution
Scarcity of annotated damaged images	Data Augmentation + Transfer Learning + Synthetic Data
Class imbalance (damaged vs. undamaged)	Focal Loss + Class Weighting + Oversampling
Detecting subtle damage	Higher input resolution + Attention Mechanisms
Limited mobile device resources	INT8 Quantization + Pruning + Knowledge Distillation
Visual similarity among 20 classes	Hard Negative Mining + Contrastive Loss
Inconsistent lighting and camera conditions	Photometric Augmentation + Domain Adaptation
Managing accumulated feedback images	Archiving system + DVC version control

Bibliography

- [1] World Bank and United Nations, “Gaza rapid damage and needs assessment,” Washington, DC, 2024, post-conflict damage and needs assessment report.
- [2] UN-Habitat, “Gaza city profile: Multi-sectoral monitoring of urban damage,” 2023.
- [3] Y. Gao and K. M. Mosalam, “Deep learning for image-based damage detection and classification of civil infrastructure,” *Computer-Aided Civil and Infrastructure Engineering*, vol. 33, no. 9, pp. 748–768, 2018.
- [4] B. Kim, J. Yu, and S. Lee, “Vision-based damage assessment of structures using deep learning,” *Automation in Construction*, vol. 136, pp. 104–181, 2022.
- [5] F. Nex, D. Duarte, F. G. Tonolo *et al.*, “Uav-based remote sensing for damage assessment in urban areas,” *International Journal of Disaster Risk Reduction*, vol. 38, pp. 101–118, 2019.
- [6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [7] X. Gao, C. Cao, and X. Yi, “Using the improved YOLOv11 model to enhance computer vision applications for building crack detection algorithms,” *Scientific Reports*, vol. 15, p. 38843, 2025.

-
- [8] J. Li, H. Zhang, Y. Wang, L. Chen, and M. Zhao, “AHD-YOLO: An adaptive hybrid dynamic network for building damage detection,” *IET Image Processing*, 2026, advance online publication.
 - [9] X. Yang, E. del Rey Castillo, Y. Zou, L. Wotherspoon, and B. Hayward, “Automated concrete bridge damage detection using an efficient vision transformer-enhanced anchor-free YOLO,” *Engineering*, vol. 51, no. 8, pp. 311–326, 2025.
 - [10] Z. Ge, X. Wang, M. Li, Y. Zhang, and H. Liu, “DDNet: Disaster damage detection for buildings based on dual-temporal joint attention network,” *Scientific Reports*, vol. 15, p. 42513, 2025.
 - [11] Y. Liu, X. Chen, Z. Wang, and S. Li, “Defect detection of prefabricated building components with integrated YOLOv5s-GhostNet,” *Concurrency and Computation: Practice and Experience*, vol. 37, no. 6, p. e70145, 2025.